TOPIC 5.6

Feature Scaling

Normalisation (min-max) and standardisation: putting features on a fair scale

WHAT YOU WILL LEARN

- Explain **why** features measured in different units must often be scaled
- Apply **min-max normalisation** and **standardisation** by hand
- Choose between them, and know how **outliers** affect each
- Know which algorithms **need** scaling and which do not
- Avoid **data leakage**: fit the scaler on the training data only

1. Why scale?

Real data mixes units. Age is between 20 and 60; salary is between 30,000 and 90,000; a car's weight is in thousands and its number of cylinders is 3 to 8. Many algorithms compare or add up the raw numbers, so the feature with the **largest numbers dominates** simply because of its units, not because it matters more.

WORKED EXAMPLE **Who is more alike?**

- Person A: age 25, salary 50,000. Person B: age 45, salary 52,000
- Distance in raw numbers = $\sqrt{(20^2 + 2000^2)}$ = **2000.1**. The age gap (20 years) is invisible next to the salary gap (2,000), because salary has bigger numbers
- After min-max scaling (age 20 to 60, salary 30,000 to 90,000 → 0 to 1): age difference = **0.500**, salary difference = **0.033**
- The 20-year age gap now counts for 15 times as much as the 2,000 salary gap: both features get a fair say

DEFINITION **Feature scaling**

Transforming numeric features so they are on **comparable scales**. The two most common methods are **normalisation (min-max scaling)** and **standardisation (z-score scaling)**.

2. Min-max normalisation

It squeezes every value into the range **0 to 1** (or another chosen range):

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

The smallest value becomes 0, the largest becomes 1, and everything else lands in between in proportion.

WORKED EXAMPLE **Example: 10, 20, 30, 40, 50**

- x_min = 10, x_max = 50, so x_max − x_min = 40
- 10 → (10 − 10)/40 = **0**, 20 → 10/40 = **0.25**, 30 → **0.5**, 40 → **0.75**, 50 → **1**

3. Standardisation (z-score)

It re-centres the values so the **mean is 0** and the **standard deviation is 1**:

$$z = \frac{x - \mu}{\sigma}$$

Here μ is the mean and σ the standard deviation. The result says *how many standard deviations a value is from the mean*: z = 2 means two standard deviations above average. The values are **not limited to a fixed range**.

WORKED EXAMPLE **Same example: 10, 20, 30, 40, 50**

- Mean μ = 30. Standard deviation σ = √200 = 14.142 (the population formula, which is what scikit-learn's StandardScaler uses)
- 10 → (10 − 30)/14.142 = **−1.414**, 20 → **−0.707**, 30 → **0**, 40 → **0.707**, 50 → **1.414**
- The new values have mean 0 and standard deviation 1

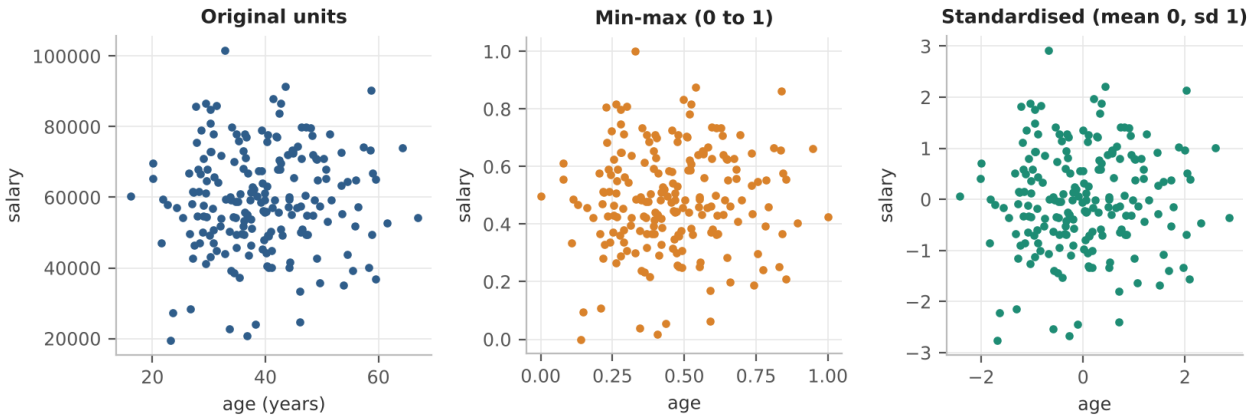


Figure 1. The same 200 people (age and salary). The shape of the cloud never changes; only the axes do. After scaling, both features span similar ranges.

	Normalisation (min-max)	Standardisation (z-score)
Formula	(x − min) / (max − min)	(x − mean) / sd
Result	Between 0 and 1	Mean 0, sd 1; no fixed range
Outliers	Strongly affected: one extreme value squeezes everyone else	Affected, but less squeezed; no fixed range
Good when	You need a bounded range (e.g. pixels, neural networks), no big outliers	Data roughly bell-shaped; many linear and distance-based models

4. What outliers do

Take six values: 10, 12, 14, 16, 18 and an outlier 100. Min-max scaling gives the outlier 1, and squeezes the five ordinary values into the narrow band 0 to 0.089: they become hard to tell apart. Standardisation keeps them in a modest range, but the outlier still pulls the mean up and inflates the standard deviation, so the ordinary values all become negative (−0.57 to −0.32).

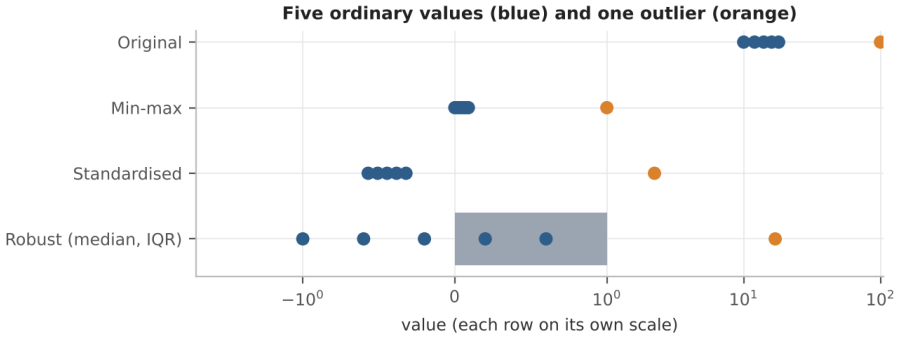


Figure 2. Where the same six values end up under each method (horizontal axis in a log-like scale). Min-max crowds the ordinary values together.

If your data has outliers, you can remove them (if they are errors), or use **RobustScaler**, which centres on the **median** and divides by the **interquartile range**, so extreme values barely move the scale.

5. Which algorithms need scaling?

Needs scaling (uses distances or gradients)	Does not need scaling
K-nearest neighbours, K-means, SVM (distances)	**Decision trees**, random forests, gradient-boosted trees (they only compare a feature with a threshold)
Linear and logistic regression trained by gradient descent (scaling makes it converge much faster: see note 5.1)	Ordinary least squares for plain linear regression gives the same predictions (coefficients just change units)
Ridge, Lasso, Elastic Net (the penalty depends on coefficient size: see 5.5)	Naive Bayes (works per feature)
PCA, neural networks	

6. Does it really matter? A test

We use the classic wine data set (178 wines, 13 chemical measurements). The feature `proline` ranges from 278 to 1680, while `nonflavanoid_phenols` ranges from 0.13 to 0.66. We train a 5-nearest-neighbours classifier on 70% of the wines and test on the other 30%.

Preparation	Test accuracy
No scaling	72.2%
Min-max scaling	96.3%
Standardisation	96.3%

Without scaling, `proline` (values in the hundreds and thousands) decides almost every distance and the other twelve measurements are ignored. Scaling gives each one a vote and lifts accuracy sharply.

7. The golden rule: fit on the training data only

> **WARNING** **Data leakage**
> The scaler learns numbers (min and max, or mean and standard deviation). If you compute them on the **whole** data set before splitting, information from the test rows leaks into training and the test score becomes over-optimistic. **Learn the numbers from the training rows only**, then apply the same numbers to the test rows.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # learn mean and sd from TRAIN only
X_test_s  = scaler.transform(X_test)        # reuse them on TEST (never fit again)
```

On the wine data, the feature means computed from the training rows differ from those computed on all rows by up to 0.09 of a standard deviation: small here, but never zero. A scikit-learn **Pipeline** (`make_pipeline(StandardScaler(), model)`) does this correctly automatically, even inside cross-validation.

Also scale only the **input features**. The **target** is left as it is (or, if you do scale it for regression, remember to convert predictions back). Do not scale 0/1 dummy columns from one-hot encoding or the target class labels in classification.

Summary

> **SUMMARY**
> - Scaling puts features on comparable scales so that large-valued features do not dominate
> - **Normalisation (min-max):** (x − min)/(max − min), range 0 to 1; sensitive to outliers
> - **Standardisation:** (x − mean)/sd; mean 0, sd 1; the usual default for linear and distance-based models
> - **RobustScaler** uses median and IQR when outliers are present
> - Needed by KNN, K-means, SVM, gradient-descent models, regularized regression, PCA; **not** needed by tree-based models
> - **Fit the scaler on the training data only**, then transform both train and test

Practice questions

Try each question before reading the answer underneath it.

Q1. Standardise the values 10, 20, 30, 40, 50. What is the z-score of 50?

Answer. $\mu = 30$, $\sigma = \sqrt{200} \approx 14.14$, so $z = (50 - 30)/14.14 \approx$ **1.41**.

Q2. Normalise 35 given a minimum of 20 and a maximum of 60.

Answer. $(35 - 20)/(60 - 20) = 15/40 =$ **0.375**.

Q3. Why do decision trees not need scaling?

Answer. A tree only asks whether a feature is above or below a threshold, and rescaling a feature does not change which rows fall on each side.

Q4. Why must the scaler be fitted on the training set only?

Answer. Fitting on all the data lets test-set information leak into training, so the test score looks better than the true performance on new data.

Q5. Why is min-max scaling risky with outliers?

Answer. The outlier defines the maximum, so every other value is squeezed into a tiny part of the 0 to 1 range and the differences between them are lost.